

DBMS Instances and Processes

DBMS Lecture Series

Unit-4

Dr. Pavinder Yadav

Assistant Professor
School of Computer Science
University of Petroleum and Energy Studies
Dehradun



- 1 DBMS Instance
- 2 DBMS Internal Memory Structure
- 3 Background Processes

1 DBMS Instance

2 DBMS Internal Memory Structure

3 Background Processes

Learning Objectives

- Understand what a **DBMS Instance** is.
- Differentiate between a **Database** and an **Instance**.
- Explore how an instance works internally (Memory + Processes).

Definition

A **DBMS Instance** is a collection of **memory structures** and **background processes** that manage access to the database files stored on disk.

Database:

- Physical data stored on disk.
- Includes data files, control files, log files.
- Static and persistent.

Instance:

- Active set of memory + processes.
- Handles queries and transactions.
- Exists only while DBMS is running.

Examples

Analogy: The database is like a **library of books**, while the instance is the **librarian** who retrieves and manages the books.

Two Major Parts

- **Memory Structures:** Buffer cache, log buffer, shared pool, etc.
- **Background Processes:** Perform I/O, recovery, and transaction control.

Key Idea

Every running DBMS server (like *mysqld* or *Oracle instance*) is an **instance**. Stopping the service terminates the instance, but the database (data files) remains.

Phases of an Instance

- ➊ **STARTUP:** Allocates memory, starts background processes.
- ➋ **MOUNT:** Reads control files and prepares access to data files.
- ➌ **OPEN:** Database available for user connections.
- ➍ **SHUTDOWN:** Writes final logs, releases memory.

Examples

In Oracle, use:

- `STARTUP` – to start instance.
- `SHUTDOWN IMMEDIATE` – to stop instance.

Single vs Multiple Instances

- Usually, one instance per database.
- In cluster systems (e.g. **Oracle RAC**), multiple instances can access the same database files.

Advantage

Multiple instances improve **scalability** and **fault tolerance** for high-availability systems.

Examples

Example: Two instances of Oracle access the same set of datafiles in a Real Application Cluster (RAC) environment.

When you start MySQL Server:

- Memory allocated: Buffer pool, log buffer, query cache.
- Threads started: I/O, log, background threads.

Commands:

- `SHOW VARIABLES LIKE 'version';`
- `SHOW ENGINE INNODB STATUS;`
- Restart service → restarts instance.

Examples

MySQL's *mysqld* process = DBMS instance running in RAM.

Key Takeaways

- A **DBMS Instance** = memory + processes managing the database.
- A **Database** = stored data files on disk.
- Instances are temporary; databases are persistent.

Checkpoint Question

Question: What happens when you stop a DBMS instance?

Answer: The data on disk remains intact, but users lose access until the instance restarts.

1 DBMS Instance

2 DBMS Internal Memory Structure

3 Background Processes

Learning Objectives

- Understand how memory is organized inside a running **DBMS instance**.
- Learn about **shared memory areas** and **private process memory**.
- Explore how caches and buffers improve performance.

Definition

The **DBMS Internal Memory Structure** refers to all the memory areas used by the DBMS to process queries, cache data, and maintain transaction logs while the instance is running.

Main Purposes

- Store data temporarily while queries execute.
- Cache frequently accessed disk blocks.
- Buffer log entries before writing to disk.
- Manage sorting, joining, and aggregation operations.

Examples

Analogy: If the database is a **bookshelf**, memory areas are the **table and notes** where we work temporarily before returning books.

Two Major Types

- **System (Shared) Memory** — Common workspace for all sessions.
- **Process (Private) Memory** — Dedicated memory per user session.

Key Difference

Shared memory enables **inter-process communication**, while private memory ensures **session isolation**.

Main Components

- **Buffer Cache (Data Cache)** – Holds copies of data blocks from disk.
- **Log Buffer** – Temporary area for transaction redo information.
- **Shared Pool** – Stores parsed SQL statements and data dictionary.
- **Large Pool / Java Pool** – Used for backups, Java procedures, or streams.

Examples

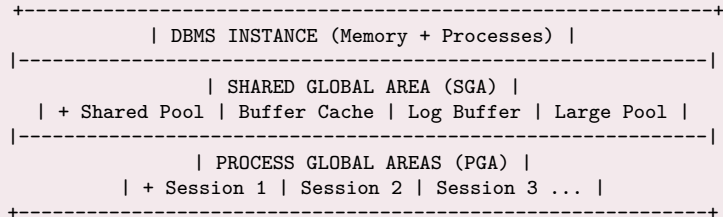
When the same query runs again, DBMS can reuse the parsed plan from the **Shared Pool** instead of reparsing it.

- **Program Global Area (PGA)** – Each process's private workspace.
- **Sort Area / Hash Area** – Temporary area for joins, GROUP BY, ORDER BY.
- **Session Memory** – Stores user variables, cursors, and state.

Examples

During a big ORDER BY, the temporary sorted data is held in the **Sort Area** before being returned to the user.

Simplified Architecture



Interpretation

The **SGA** is common for all connections; each user has their own **PGA**.

Key Memory Areas:

- **Buffer Pool** – caches data and index pages.
- **Redo Log Buffer** – stores redo entries before flushing to log files.
- **Sort / Join Buffers** – created per session.

Useful Commands:

- `SHOW VARIABLES LIKE 'innodb_buffer_pool_size';`
- `SHOW ENGINE INNODB STATUS\G;`

Examples

These commands reveal how much memory MySQL allocates for caching and transaction logs.

Why Memory Matters

- Reduces **disk I/O** by caching frequently accessed data.
- Enables **faster query execution** through optimized sorting and joins.
- Ensures **durability** via log buffering before commit.

Common Issue

If the buffer cache is too small, the system experiences **cache misses** and frequent disk reads, slowing down performance.

In MySQL:

- `SHOW ENGINE INNODB STATUS\G;`
- View buffer pool and background thread stats.

In Oracle:

- `SHOW SGA;`
- Displays total shared memory usage and cache sizes.

Examples

Students can compare the memory allocation of MySQL and Oracle to understand real-world implementation of DBMS instances.

Key Takeaways

- Memory structures are divided into **shared** and **private** areas.
- Components such as Buffer Cache, Log Buffer, and Shared Pool boost performance.
- Proper memory sizing is crucial for efficiency and durability.

Checkpoint Question

Question: What happens when the buffer cache becomes full? **Answer:** DBMS replaces old blocks using an LRU (Least Recently Used) policy, possibly causing additional disk I/O.

1 DBMS Instance

2 DBMS Internal Memory Structure

3 Background Processes

Learning Objectives

- Understand what **background processes** are in a DBMS.
- Learn their purpose in managing memory and disk I/O.
- Identify major background processes in Oracle and MySQL.

Definition

Background Processes are helper programs or threads that continuously run behind the scenes to manage memory structures, input/output (I/O), and transaction consistency in a DBMS instance.

Key Responsibilities

- Perform **disk I/O operations**.
- Manage **transactions and recovery**.
- Maintain **buffer caches and log buffers**.
- Ensure **data consistency and durability**.

Examples

Analogy: If the DBMS instance is a **factory**, background processes are the **workers and supervisors** who keep machines (memory) running efficiently.

Essential Oracle Processes

- **DBWR (Database Writer)** — Writes modified data blocks from the buffer cache to datafiles.
- **LGWR (Log Writer)** — Writes redo entries from the log buffer to redo log files.
- **SMON (System Monitor)** — Performs instance recovery at startup and cleans up temporary segments.
- **PMON (Process Monitor)** — Cleans up failed user processes and releases their resources.
- **CKPT (Checkpoint Process)** — Coordinates checkpoints by signaling DBWR and updating control files.

Key Idea

These five processes form the **core engine** of every Oracle instance and are responsible for ensuring ACID properties.

- **ARCn** – Archives redo log files for backup.
- **RECO** – Resolves distributed transaction failures.
- **MMON** / **MMNL** – Monitor memory and performance.
- **CJQn** – Coordinates scheduled jobs.

Examples

For performance tuning, Oracle's **MMON** gathers statistics used by the optimizer and reports performance metrics.

Key Threads

- **Master Thread** – Handles flushing of dirty pages and writes logs to disk.
- **I/O Threads** – Perform asynchronous read and write requests.
- **Log Thread** – Writes redo logs from buffer to disk.
- **Purge Thread** – Removes obsolete undo entries.
- **Page Cleaner Thread** – Assists in writing dirty pages to free cache space.

Examples

In MySQL, run: `SHOW ENGINE INNODB STATUS\G`; to observe these background threads in action.

Mapping

Memory Component	Maintained By
Buffer Cache	DBWR / Master Thread
Log Buffer	LGWR / Log Thread
Shared Pool	SMON / PMON
Sort Area	User Process

Examples

Each background process serves a **specific memory area**, ensuring data integrity and optimized access.

When User Executes an UPDATE

- 1 Data block read into **Buffer Cache**.
- 2 Update performed in memory.
- 3 Redo entry recorded in **Log Buffer**.
- 4 **LGWR** writes redo → Redo Log Files.
- 5 **DBWR** later writes data block → Datafile.
- 6 **CKPT** ensures data consistency at checkpoints.

Key Principle

Follows **Write-Ahead Logging (WAL)** — logs are written before actual data for recovery safety.

Oracle:

- `SELECT p.spid, p.program FROM v$process;`
- Lists system-level background processes.

MySQL:

- `SHOW ENGINE INNODB STATUS\G;`
- Displays current thread activities.

Examples

Students can compare Oracle processes (LGWR, DBWR) with MySQL threads to understand internal concurrency.

Key Takeaways

- Background processes handle all behind-the-scenes tasks.
- Oracle uses named processes; MySQL uses background threads.
- Processes coordinate for durability, consistency, and recovery.

Checkpoint Question

Why doesn't DBWR write data immediately after every update?

Because it performs **lazy writes** to reduce I/O overhead; meanwhile, **LGWR** ensures durability through redo logs.

Thank You!



DBMS Instances and Processes

DBMS Lecture Series

Unit-4

Dr. Pavinder Yadav

Assistant Professor
School of Computer Science
University of Petroleum and Energy Studies
Dehradun



- 1 Data Types in DBMS
- 2 Roles and Privileges in DBMS

1 Data Types in DBMS

2 Roles and Privileges in DBMS

Learning Objectives

- Understand the role of **data types** in defining table columns.
- Learn various categories of SQL data types.
- Explore implementation examples from **MySQL** and **Oracle**.

Definition

A **Data Type** defines the kind of values a column can store in a database table, such as **numbers**, **text**, **dates**, or **binary data**.

Why Data Types Matter

- Enforce **data integrity** by restricting allowed values.
- Determine **storage size** and **memory allocation**.
- Influence **query performance** and indexing behavior.
- Ensure consistent **data interpretation** across applications.

Examples

A column declared as INT will reject alphabetic input, ensuring integrity of numeric data.

SQL Standard Data Type Groups

- **Numeric Types** – store integers, decimals, floating-point numbers.
- **Character/String Types** – store fixed or variable-length text.
- **Date/Time Types** – store temporal values.
- **Boolean Type** – store logical values TRUE/FALSE.
- **Binary / Large Object Types** – store images, documents, multimedia.

Example

```
CREATE TABLE Student( RollNo INT, Name VARCHAR(30), DOB DATE, Grade  
CHAR(2));
```

Common Numeric Types

- **INT / INTEGER** – Whole numbers.
- **DECIMAL(p,s)** – Fixed precision numbers (e.g., 123.45).
- **FLOAT / DOUBLE** – Approximate decimal values.
- **SMALLINT / BIGINT** – Smaller or larger integer ranges.

Examples

Salary **DECIMAL(10,2)** → Allows up to 10 digits, 2 after decimal. Age **INT CHECK (Age > 0)**;

Types of Character Data

- **CHAR(n)** – Fixed-length string (padded with spaces).
- **VARCHAR(n)** – Variable-length string.
- **TEXT / CLOB** – Large blocks of text.

Example

Name `CHAR(10)` always stores 10 bytes. Address `VARCHAR(100)` stores only the actual number of characters used.

Examples

In MySQL: `VARCHAR(255)` is often used for names, titles, or descriptions.

Common Temporal Types

- **DATE** – Stores date only (YYYY-MM-DD).
- **TIME** – Stores time only (HH:MM:SS).
- **DATETIME / TIMESTAMP** – Stores both date and time.
- **YEAR** – Stores a year value.

Examples

```
DOB DATE → '2001-05-16' created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP;
```

Boolean Data Type

- Stores **TRUE/FALSE** or logical 0/1.
- MySQL internally stores as TINYINT(1).

Binary and Large Object Types

- **BINARY / VARBINARY(n)** – Fixed or variable binary strings.
- **BLOB (Binary Large Object)** – Stores images, PDFs, videos.
- **CLOB (Character Large Object)** – Large text documents.

Examples

```
Photo BLOB; isActive BOOLEAN DEFAULT TRUE;
```

MySQL Examples

- INT, DECIMAL, VARCHAR, TEXT, DATE
- TINYINT(1) used for Boolean values.
- BLOB / LONGBLOB for binary data.

Oracle Examples

- NUMBER(p,s), VARCHAR2(n), DATE
- CLOB / BLOB for large text or binary data.
- BOOLEAN not supported in SQL layer, only in PL/SQL.

Key Difference

Oracle uses VARCHAR2, while MySQL uses VARCHAR.

Best Practices

- Use smallest data type sufficient for expected values.
- Avoid excessive precision or length.
- Match data type with usage (numeric vs string comparisons).
- Use CHECK constraints for validation.

Examples

Salary DECIMAL(8,2) instead of FLOAT for accuracy. Phone VARCHAR(10) instead of BIGINT to preserve leading zeros.

Key Takeaways

- Data types ensure integrity, optimize storage, and improve query speed.
- Choose types based on **data nature and expected operations**.
- Both MySQL and Oracle support similar, but not identical, type sets.

Checkpoint Question

Why should phone numbers not be stored as numeric data type?

Because phone numbers may contain leading zeros and symbols like “+”, which are not numeric values.

1 Data Types in DBMS

2 Roles and Privileges in DBMS

Learning Objectives

- Understand the concepts of **user roles** and **privileges** in a DBMS.
- Learn how access control ensures data security.
- Explore SQL commands for granting and revoking permissions.

Definition

Roles and Privileges define the **authorization mechanism** in DBMS that determines *who* can access *what*, and *how*.

Why Authorization is Needed

- Prevents unauthorized access to sensitive data.
- Ensures different users have **appropriate levels of access**.
- Protects data from accidental or malicious modification.
- Maintains accountability and auditing of user actions.

Examples

For example, a student user should not be able to **modify** marks, but an instructor should be allowed to **update** them.

Types of Privileges

- **System Privileges:** Rights to perform administrative tasks.
- **Object Privileges:** Rights to access or modify specific tables, views, or sequences.

Example

System Privilege: CREATE TABLE, CREATE USER

Object Privilege: SELECT, INSERT, UPDATE, DELETE

Examples

```
GRANT SELECT, INSERT ON Student TO user1;
```

Syntax

```
GRANT privilege_list ON object_name TO user_list;
```

Purpose

- Used by the **DBA or owner** to give privileges to users.
- Allows both direct and role-based permission assignment.

Examples

```
GRANT SELECT, UPDATE ON Employee TO hr_team;  
GRANT ALL PRIVILEGES ON Company TO admin;
```

Syntax

```
REVOKE privilege_list ON object_name FROM user_list;
```

Purpose

- Used to **remove privileges** previously granted.
- Ensures tighter security when a user no longer requires access.

Examples

```
REVOKE UPDATE ON Employee FROM user1; REVOKE ALL PRIVILEGES ON Student FROM intern;
```

Definition

A **Role** is a **named group of privileges** that can be granted to users collectively, simplifying management.

Purpose of Roles

- Reduces administrative overhead.
- Allows grouping of privileges by function or job responsibility.
- Makes privilege management scalable in large organizations.

Examples

```
CREATE ROLE manager;  
GRANT SELECT, UPDATE ON Employee TO manager;  
GRANT manager TO user1;
```

Common SQL Commands

- `CREATE ROLE role_name;`
- `GRANT privilege_list TO role_name;`
- `GRANT role_name TO user;`
- `REVOKE role_name FROM user;`

Examples

```
CREATE ROLE faculty;  
GRANT INSERT, UPDATE ON Student  
TO faculty;  
GRANT faculty TO satyam;
```

Illustration

```
+-----+
| DBA |
| → Has all system privileges |
| → Can create roles/users |
+-----+
↓
+-----+
| ROLE: Instructor |
| → SELECT, UPDATE ON Marks |
+-----+
↓
+-----+
| USER: Faculty1 |
| → Granted role Instructor |
+-----+
```

Observation

Roles simplify administration by **grouping multiple privileges** and assigning them as a single unit.

Key Points

- Revoking a role automatically removes all privileges under it.
- Individual privileges can be revoked selectively.
- Roles can be made **default** or **temporary**.

Examples

```
REVOKE faculty FROM satyam;  
REVOKE UPDATE ON Student FROM faculty;
```

Oracle:

- Fully supports roles with fine-grained privileges.
- Allows enabling/disabling roles per session.
- Example: SET ROLE manager;

MySQL:

- Introduced roles from version 8.0.
- Roles must be explicitly activated with SET ROLE.
- Example: SET DEFAULT ROLE manager TO user1;

Note

Both Oracle and MySQL use roles to simplify **access control administration**.

Key Takeaways

- Privileges define what actions users can perform.
- Roles group multiple privileges for easier management.
- GRANT and REVOKE are fundamental authorization commands.

Checkpoint Question

What is the difference between a privilege and a role?

A **privilege** is a specific right; a **role** is a collection of privileges.

Thank You!



DBMS Query Processing

DBMS Lecture Series

Unit-4

Dr. Pavinder Yadav

Assistant Professor
School of Computer Science
University of Petroleum and Energy Studies
Dehradun



- 1 Introduction to Query Processing
 - Parsing & Validation
 - Translation to Relational Algebra
 - Logical Rewriting (Equivalences)
 - Physical Planning (High-Level Intuition)
 - Execution Model
- 2 Examples
- 3 Translating SQL Queries into Relational Algebra

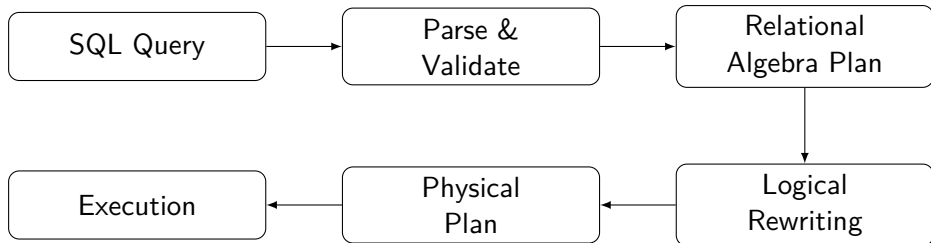
- 1 Introduction to Query Processing
 - Parsing & Validation
 - Translation to Relational Algebra
 - Logical Rewriting (Equivalences)
 - Physical Planning (High-Level Intuition)
 - Execution Model
- 2 Examples
- 3 Translating SQL Queries into Relational Algebra

- SQL is **declarative**: we say *what* we want, not how to get it.
- The DBMS converts SQL into an efficient execution plan.
- Goal: **Correct** results, **Fast** execution, **Scalable** to big data.

Remark

Query processing bridges the gap between high-level SQL and low-level data access.

- Parsing & Validation
- Translation to Relational Algebra (Logical Plan)
- Logical Rewriting (Equivalence Rules)
- Physical Plan Selection (Cost-Based)
- Execution (Iterators/Operators)



- Understand the end-to-end flow from SQL to results.
- Translate SQL into relational algebra (RA) quickly.
- Recognize common logical rewrites (**predicate** & **projection** pushdowns).
- Read a plan and identify major **access paths** and **join algorithms**.

Important

We keep cost-model math *light*; focus on the **intuitions** that influence plan choices.

- **Parsing:** Check SQL grammar; build a parse tree.
- **Validation:** Resolve names (tables/columns), types, and privileges.
- **Output:** A syntactically correct, semantically valid internal form.

Examples

```
SELECT name, salary FROM Emp WHERE salary > 100000;  
Check: table Emp? columns name, salary? types ok?
```

Common Errors Caught Early

- Misspelled table/column names
- Type mismatches (e.g., comparing string to integer)
- Missing privileges
- Keep in mind: if SQL fails here, optimizer never runs. Clean inputs matter

Relational Algebra (RA): The Logical View

- Core operators: σ (select/filter), π (project/columns), $\bowtie$ (join)
- Others: γ (group-by/aggregate), $\cup, \cap, -$ (set ops), rename
- RA expresses **what** sub-steps are needed to compute the query.

Example

SQL $\rightarrow$ RA: Simple Filter

SQL

```
SELECT name  
FROM Emp  
WHERE salary > 100000;
```

RA

$\pi_{\text{name}}(\sigma_{\text{salary} > 100000}(\text{Emp}))$

Remark

σ reduces rows (filters), π reduces columns.

SQL

```
SELECT e.name, d.dept_name
FROM Emp e JOIN Dept d
ON e.dept_id = d.id
WHERE e.city = 'Pune';
```

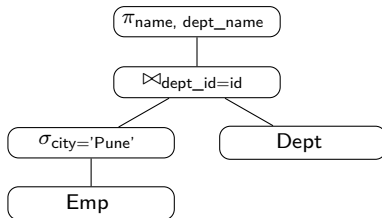
RA

$$\pi_{\text{name, dept_name}}((\sigma_{\text{city}='Pune'}(\text{Emp})) \bowtie_{\text{dept_id=id}} \text{Dept})$$

Examples

Push the filter σ down onto Emp to shrink data *before* join.

RA Trees



Why Rewrite?

Predicate and projection pushdown are data processing optimizations that speed up queries by moving filtering and column selection closer to the data source. Predicate pushdown reduces the number of rows fetched by applying filtering conditions (predicates) at the storage level, while projection pushdown reduces the number of columns by only reading the columns specified in the query. Both techniques decrease the amount of data transferred between the data source and the processing engine, improving performance.

- Many RA expressions are **equivalent** (same answer).
- Pick the form that **does less work**.
- Classic rules: predicate pushdown, projection pushdown, join reordering.

Important

Smaller intermediate results = fewer I/Os, less CPU, faster queries.

Before

$$\pi_{A,B}(\sigma_{C>10}(R \bowtie S))$$

After

$$\pi_{A,B}((\sigma_{C>10}(R)) \bowtie \pi_{\text{needed cols}}(S))$$

Note

Push filters and column trimming *as low as possible*.

Join Reordering (Associativity/Commutativity)

- $(R \bowtie S) \bowtie T \equiv R \bowtie (S \bowtie T)$
- $R \bowtie S \equiv S \bowtie R$

Examples

Start joins with the most selective/ smallest intermediate to reduce cost.

4. Physical Planning (High-Level Intuition)

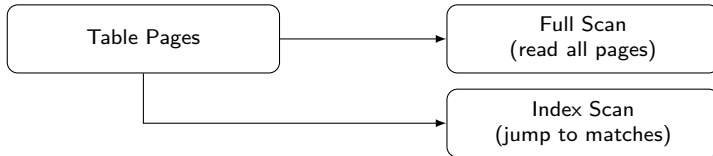
From Logical to Physical

- Choose **access paths**: full scan vs. index scan.
- Choose **join algorithms**: Nested Loop, Index NLJ, Sort-Merge, Hash Join.
- Consider **statistics**: table size, distinct counts, histograms.

Remark

Cost-based optimizers estimate I/O and CPU to pick a low-cost plan.

Access Paths



Examples

Use index scan for `WHERE salary > 100000` if a B⁺-tree on salary exists.

- **Nested Loop Join (NLJ)**: simple; good if outer is small.
- **Index Nested Loop (INLJ)**: great if inner has index on join key.
- **Sort-Merge Join (SMJ)**: if inputs are sorted (or can be sorted efficiently).
- **Hash Join (HJ)**: build hash on smaller side; strong for large equi-joins.

Mini Decision Heuristics

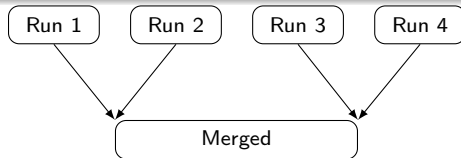
- Highly selective filter + index on inner: **INLJ**.
- Large equi-join, no helpful index: **Hash Join**.
- Already sorted on join key: **Sort-Merge Join**.
- Otherwise: NLJ/Block-NLJ as baseline (may be slow for big data).

External sorting is a class of sorting algorithms that can handle massive amounts of data. External sorting is required when the data being sorted do not fit into the main memory of a computing device (usually RAM) and instead they must reside in the slower external memory, usually a disk drive.

- Data may not fit in RAM; sort in **passes**.
- Phase 1: create sorted **runs** in memory and write to disk.
- Phase 2: **k-way merge** runs until one sorted file remains.

Important

Used by ORDER BY, DISTINCT, sort-merge joins, and some set ops.



The execution model in DBMS query processing defines how a database management system executes a query plan, dictating the method of data flow between operators and the evaluation strategy. Key execution models include:

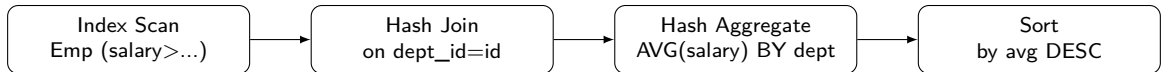
Iterator (Volcano) Model

- Each operator supports `open()`, `next()`, `close()`.
- Pipelining**: parent consumes tuples as child produces them.
- Materialization**: some ops (sort, hash build) must spill or store.

Note

Streaming results early improves perceived latency and memory use.

Operator Pipeline



Materialization Model:

- In this model, an operator fully computes its output and stores it in a temporary data structure (e.g., a temporary table or file) before passing it to the next operator in the plan.
- This can be beneficial when an operator's output is needed multiple times or when the output size is manageable.
- However, it can lead to higher memory or disk I/O overhead for large intermediate results.

Vectorized/Batch Model:

- This model processes data in batches or vectors of tuples, rather than one tuple at a time.
- Operators receive and process entire batches of data, which can significantly improve performance by reducing function call overhead and enabling more efficient use of CPU caches and vectorized instructions.
- This model is particularly well-suited for analytical workloads (OLAP) and data warehousing where large scans and aggregations are common. SQL Server's Batch Mode execution is an example of this.

These models represent different trade-offs in terms of performance, memory usage, and suitability for various types of database workloads. Most modern DBMSs employ a combination of these models or variations thereof to optimize query execution.

- Identify access paths: **Seq Scan** vs **Index Scan**.
- Identify join type: NLJ / INLJ / SMJ / HJ.
- Look for **filters pushed down**, projections trimmed, and order/group ops.

Examples

If you see Index Only Scan, the index covers all needed columns.

- 1 Introduction to Query Processing
 - Parsing & Validation
 - Translation to Relational Algebra
 - Logical Rewriting (Equivalences)
 - Physical Planning (High-Level Intuition)
 - Execution Model
- 2 Examples
- 3 Translating SQL Queries into Relational Algebra

SQL: SELECT name FROM Emp WHERE salary > 100000;

- RA: $\pi_{\text{name}}(\sigma_{\text{salary} > 100000}(\text{Emp}))$
- Physical: Index range scan on salary if available; else seq scan.

Remark

Project only needed columns to reduce tuple width early.

SQL: SELECT e.name, d.dept_name FROM Emp e JOIN Dept d ON e.dept_id=d.id
WHERE e.city='Pune';

- Push $\sigma_{\text{city}='Pune'}$ to Emp.
- If Dept is small or Orders not involved, INLJ with index on e.dept_id or HJ depending on stats.

Important

Start with the **most selective** relation to reduce join cost.

```
SQL: SELECT d.dept_name, AVG(e.salary) AS avg_sal  
FROM Emp e JOIN Dept d ON e.dept_id=d.id  
GROUP BY d.dept_name  
ORDER BY avg_sal DESC;
```

- Filter early if possible; join; then **Hash Aggregate** or Sort+Aggregate.
- Final **Sort** on avg_sal DESC.

- **Avoid** wrapping indexed columns in functions (`YEAR(dt)=...`) unless function-based index exists.
- Prefer `dt BETWEEN ...` or `col > constant`.

S

sargable predicates let the optimizer use **indexes** effectively.

Indexing Essentials

- Index WHERE, JOIN, GROUP BY, ORDER BY keys (not everything).
- Consider **covering indexes** for hot queries.
- Keep **statistics** up-to-date for good cardinality estimates.

- Q: When is a full table scan better than an index scan?
- A: When the predicate is **not selective** (returns a large fraction) or no useful index exists.

Examples

$\sigma_{\text{salary} > 0}$ on a table where nearly all salaries are positive.

- Q: Which join first: $A \bowtie B \bowtie C$ if σ on A keeps 1%?
- A: Start with $\sigma(A)$ to shrink early, then join to others.
- Q: Why does the optimizer sometimes choose Hash Join over Index NLJ?
- A: Large inputs with no selective outer or missing/low-quality index on inner; HJ can be cheaper overall.

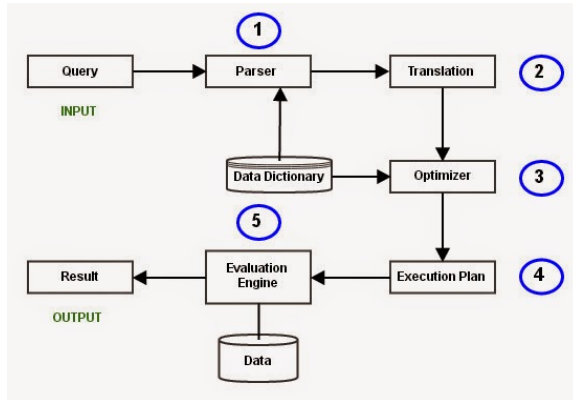


Figure: Query Processing

- SQL → RA → Rewrites → Physical Plan → Execution.
- Push **filters** and **projections** down.
- Choose access paths & join algorithms based on **selectivity** and **indexes**.
- External sort enables ORDER BY, DISTINCT, and SMJ.

Important

Think in terms of **reducing intermediate data** to save I/O and CPU.

- Deeper dive: translating more SQL patterns to RA.
- External sorting algorithms and costs (2-pass, multi-pass).
- Detailed algorithms for SELECT, JOIN, PROJECT, SET, Aggregates, Outer Joins.

- 1 Introduction to Query Processing
 - Parsing & Validation
 - Translation to Relational Algebra
 - Logical Rewriting (Equivalences)
 - Physical Planning (High-Level Intuition)
 - Execution Model
- 2 Examples
- 3 Translating SQL Queries into Relational Algebra

Learning Objectives

- Understand how SQL queries are transformed into **Relational Algebra (RA)** expressions.
- Learn standard RA symbols and their SQL equivalents.
- Practice translating SQL queries into algebraic form.

Key Idea

SQL is a **declarative language** (what to do), while **Relational Algebra** is procedural (how to do it). The DBMS converts $\text{SQL} \rightarrow \text{RA} \rightarrow \text{Execution Plan}$.

Purpose of Translation

- Provides a formal representation for query optimization.
- Enables the DBMS to apply algebraic transformations.
- Acts as an intermediate step before creating the query plan.

Examples

Example:

`SELECT name FROM Student WHERE dept = 'CSE';`

Translates to:

$$\pi_{\text{name}}(\sigma_{\text{dept}='CSE'}(\text{Student}))$$

Basic Operators

- **Selection (σ)** – Filters tuples (rows).
- **Projection (π)** – Selects attributes (columns).
- **Union ($\cup$)** – Combines tuples from two relations.
- **Set Difference ($-$)** – Tuples in one but not the other.
- **Cartesian Product ($\times$)** – All combinations of tuples.
- **Rename (ρ)** – Renames relation or attributes.

Examples

Example:

```
SELECT name FROM Student;
```

$\Rightarrow \pi_{\text{name}}(\text{Student})$

Common Derived Operators

- **Join ($\bowtie$)** – Combines relations based on condition.
- **Intersection ($\cap$)** – Tuples common to both.
- **Division ($\div$)** – “For all” queries.
- **Natural Join ($\bowtie$)** – Auto-join on common attributes.
- **Theta Join ($\bowtie_{\theta}$)** – Join using a specific condition.

Example

Student_{Student.dept_id = Department.dept_id} Department

General Translation

- **FROM** → Base relations.
- **WHERE** → Selection (σ).
- **SELECT** → Projection (π).
- **JOIN / ON** → Join ($\bowtie$).
- **GROUP BY** → Grouping (ρ).
- **HAVING** → Selection on groups.
- **UNION / INTERSECT / MINUS** → Set operations.

Examples

```
SELECT name, dept FROM Student WHERE marks > 80;
```

```
⇒  $\pi_{\text{name, dept}}(\sigma_{\text{marks} > 80}(\text{Student}))$ 
```

Examples

SQL:

```
SELECT name, rollno FROM Student WHERE dept='CSE' AND marks>70;
```

Relational Algebra:

$$\pi_{\text{name, rollno}}(\sigma_{\text{dept}='CSE' \wedge \text{marks}>70}(\text{Student}))$$

Note

Logical operators AND ($\wedge$), OR ($\vee$), and NOT ($\neg$) are valid within selection conditions.

Examples

SQL:

```
SELECT s.name, d.dept_name  
FROM Student s, Department d  
WHERE s.dept_id = d.dept_id;
```

Relational Algebra:

$$\pi_{s.name, d.dept_name}(\sigma_{s.dept_id = d.dept_id}(Student \bowtie Department))$$

Optimized (Natural Join):

$$\pi_{name, dept_name}(Student \bowtie Department)$$

Examples

SQL:

```
SELECT s.name, d.dept_name  
FROM Student s JOIN Department d  
ON s.dept_id = d.dept_id  
WHERE d.location = 'Block-A';
```

Relational Algebra:

$$\pi_{\text{name, dept_name}}(\sigma_{\text{location}='Block-A'}(\text{Student} \bowtie_{\text{Student.dept_id} = \text{Department.dept_id}} \text{Department}))$$

SQL Query

```
SELECT name FROM Student  
WHERE dept_id IN (SELECT dept_id FROM Department WHERE location='Block-A');
```

Relational Algebra Translation

$$\pi_{\text{name}}(\sigma_{\text{dept_id}} (\pi_{\text{dept_id}}(\sigma_{\text{location}='Block-A'}(\text{Department}))) (\text{Student}))$$

Examples

The inner query computes the set of departments in Block-A; the outer query filters students belonging to those departments.

Examples

SQL:

```
SELECT dept_id, AVG(marks)
FROM Student GROUP BY dept_id HAVING AVG(marks)>80;
```

Relational Algebra:

$$\gamma_{\text{dept_id, AVG(marks)}}(\sigma_{\text{AVG(marks)} > 80}(\text{Student}))$$

Note

γ (gamma) denotes **grouping and aggregation**.

UNION

```
SELECT rollno FROM Student_2024  
UNION  
SELECT rollno FROM Student_2025;
```

$$\pi_{\text{rollno}}(\text{Student_2024}) \cup \pi_{\text{rollno}}(\text{Student_2025})$$

SET DIFFERENCE

```
SELECT rollno FROM Student  
MINUS SELECT rollno FROM Alumni;
```

$$\pi_{\text{rollno}}(\text{Student}) - \pi_{\text{rollno}}(\text{Alumni})$$

Examples

SQL:

```
SELECT E1.name, E2.name  
FROM Employee E1, Employee E2  
WHERE E1.manager_id = E2.emp_id;
```

Relational Algebra:

$$\pi_{E1.name, E2.name}(\sigma_{E1.manager_id = E2.emp_id}(E1(Employee)E2(Employee)))$$

Note

(rho) ρ is used to **rename** relations in self-joins.

Example

```
SELECT name FROM Student WHERE marks > 75;
```

Relational Algebra

$$\pi_{\text{name}}(\sigma_{\text{marks} > 75}(\text{Student}))$$

Examples

Query Tree:

```
  π_name
  |
σ_marks>75
  |
Student
```

Why It Matters

- Provides a structured form for **query optimization**.
- Allows reordering and simplification of operations.
- Serves as input to the **query execution plan generator**.

Example: Selection Pushdown

$$\pi_{\text{name}}(\sigma_{\text{dept}=\text{'CSE'}}(\text{Student} \bowtie \text{Department})) \Rightarrow \pi_{\text{name}}((\sigma_{\text{dept}=\text{'CSE'}}(\text{Student})) \bowtie \text{Department})$$

Earlier filtering improves performance.

Key Takeaways

- SQL is internally represented in **Relational Algebra**.
- $\sigma \rightarrow$ Selection, $\pi \rightarrow$ Projection, $\rightarrow$ Join, $\rightarrow$ Grouping.
- Translation helps DBMS optimize and execute efficiently.

Practice Exercise

Translate into RA: SELECT E.name, D.dept_name
FROM Employee E JOIN Department D
ON E.dept_id=D.dept_id
WHERE E.salary>50000;

$\pi_{E.name, D.dept_name}(\sigma_{E.salary > 50000}(Employee \bowtie_{E.dept_id = D.dept_id} Department))$

Thank You!



DBMS Query Processing

DBMS Lecture Series

Unit-4

Dr. Pavinder Yadav

Assistant Professor
School of Computer Science
University of Petroleum and Energy Studies
Dehradun



- 1 Algorithms for External Sorting
- 2 Algorithms for SELECT Operations
- 3 Algorithms for JOIN Operations

- 1 Algorithms for External Sorting
- 2 Algorithms for SELECT Operations
- 3 Algorithms for JOIN Operations

Learning Objectives

- Understand why sorting large relations requires **external sorting**.
- Learn algorithms for **two-way** and **multi-way** merge sort.
- Analyze buffer usage and I/O cost.

Definition

External Sorting refers to sorting data that cannot fit entirely in main memory, using disk files and limited buffers.

Need for External Sorting

- Database relations are often **larger than available RAM**.
- Sorting must be performed using **disk I/O** and **limited memory buffers**.
- Used in implementing **ORDER BY**, **GROUP BY**, **DISTINCT**, and **JOIN**.

Examples

Example: If a 10 GB file must be sorted using 100 MB of memory, data must be divided into **chunks (runs)** and merged later.

Two Main Phases

① **Sorting (Run Generation)**

- Divide file into chunks that fit in memory.
- Sort each chunk and write back to disk.

② **Merging (Combine Sorted Runs)**

- Merge sorted runs repeatedly until one sorted file remains.

Examples

If 12 blocks and 3 memory buffers $\rightarrow$ create 4 sorted runs (each of 3 blocks) $\rightarrow$ merge until one remains.

Given: 12 blocks, 3 memory buffers. **Step 1: Run Generation**

- Load 3 blocks, sort, write back → Run 1
- Repeat → Run 2, Run 3, Run 4

Step 2: Merging

- Merge two runs at a time: $R1 + R2 \rightarrow R12$, $R3 + R4 \rightarrow R34$
- Merge $R12 + R34 \rightarrow$ Final sorted run

Note

Pass 0: Run generation **Pass 1,2:** Merging → Total = 3 passes

Algorithm Steps

- 1 Divide the file into runs that fit in memory.
- 2 Sort each run internally.
- 3 Merge two runs at a time in each pass.
- 4 Continue until a single sorted run remains.

Examples

For 4 runs (R1, R2, R3, R4):

- Pass 1: $R1+R2 \rightarrow R12$, $R3+R4 \rightarrow R34$
- Pass 2: $R12+R34 \rightarrow \text{Final Sorted File}$

Cost Formulas

- Let N = total blocks in file, B = buffer pages.
- Number of Passes = $1 + \lceil \log_2(N/B) \rceil$
- Each pass reads and writes all N blocks.
- Total I/O Cost = $2N \times (\text{Number of Passes})$

Examples

Example: $N = 12, B = 3 \Rightarrow \text{Passes} = 1 + \lceil \log_2(12/3) \rceil = 3 \Rightarrow \text{I/O Cost} = 2 \times 12 \times 3 = 72$ block transfers.

Motivation

Two-way merging needs multiple passes. We can reduce passes by merging **more than 2 runs** at a time.

Algorithm Steps

- 1 In each merge pass, merge up to $(B - 1)$ runs simultaneously.
- 2 Use one buffer for output.
- 3 Repeat until one sorted file remains.

Key Formula

$$\text{Number of Passes} = 1 + \lceil \log_{(B-1)}(N/B) \rceil$$

Given

File size = 12 blocks, Memory = 4 buffers ($B = 4$) Can merge $(B - 1) = 3$ runs at once.

Examples

Pass 0 (Run Generation): 12 blocks $\rightarrow$ 3 sorted runs (R1, R2, R3) each of 4 blocks.

Pass 1 (Merging): Merge all 3 runs $\rightarrow$ Final Sorted File.

Only **2 passes** (vs. 3 in two-way merge).

Conclusion

Fewer passes Lower I/O cost Better performance.

Memory Usage in Merging

- One input buffer per run.
- One output buffer.
- $\Rightarrow$ Maximum runs merged per pass = $(B - 1)$

Examples

Algorithm	Buffers (B)	Runs Merged	Passes
Two-way Merge	3	2	More
Four-way Merge	5	4	Fewer
Eight-way Merge	9	8	Much fewer

Idea

Use a **priority queue (min-heap)** to generate longer initial runs.

How It Works

- Maintain B pages in memory.
- Output smallest tuple; replace it with next input record.
- If new record $<$ last output, start a new run.

Examples

Average initial run length $2B$ instead of B . Fewer runs $\rightarrow$ Fewer merge passes.

General Formula

$$\text{Total I/O} = 2N \times (1 + \lceil \log_{(B-1)}(N/B) \rceil)$$

Examples

Example: A relation of 1000 pages, memory = 10 buffers.

$$\text{Passes} = 1 + \lceil \log_9(1000/10) \rceil = 3$$

$$\text{I/O} = 2 \times 1000 \times 3 = 6000 \text{ block transfers.}$$

Observation

Multi-way merging reduces total passes and overall I/O time.

Key Uses

- Sorting relations for **ORDER BY**, **GROUP BY**, and **DISTINCT**.
- Required for **Sort-Merge Join**.
- Often the most I/O intensive DBMS operation.

DBMS Implementation Notes

- Uses **double buffering** to overlap I/O and computation.
- Employs **replacement selection** for longer runs.
- May perform **parallel merges** on multiple disks.

Key Takeaways

- External sorting is used when data **does not fit in memory**.
- Two-way merge merges 2 runs per pass; Multi-way merges $(B - 1)$ runs.
- Fewer passes $\rightarrow$ lower I/O cost.
- Replacement selection extends run length $\approx 2B$.

Checkpoint Question

Why can't DBMS use quicksort for sorting large relations?

Because quicksort needs random access to all data in memory, while DBMS sorting must work with limited buffers and disk I/O.

- 1 Algorithms for External Sorting
- 2 Algorithms for SELECT Operations
- 3 Algorithms for JOIN Operations

Definition

The **Selection operator** $\sigma_{\text{condition}}(R)$ retrieves tuples from relation R that satisfy a given predicate.

SQL Correspondence

```
SELECT * FROM Employee WHERE dept = 'HR';  
 $\Rightarrow \sigma_{\text{dept}='HR'}(\text{Employee})$ 
```

Learning Objectives

- Understand how DBMS executes the **SELECT** (σ) operation.
- Explore various algorithms for performing selection.
- Compare their cost and applicability.

Key Influencing Factors

- **File Organization:** heap, sorted, indexed, hashed.
- **Predicate Type:** equality, range, composite.
- **Index Availability:** clustered vs non-clustered.
- **Selectivity:** fraction of tuples satisfying condition.

Examples

```
SELECT * FROM Student WHERE dept='CSE' AND marks>80;  
→ DBMS chooses plan based on indexes on dept or marks.
```

Common Algorithms

- 1 Linear (Sequential) Scan
- 2 Binary Search
- 3 Index Scan (B^+ -tree or Hash)
- 4 Hash-based Search
- 5 Composite Conditions (AND, OR, NOT)

Examples

Example:

```
SELECT * FROM EMPLOYEE WHERE SALARY > 60000;  
 $\Rightarrow \sigma_{\text{salary} > 60000}(\text{EMPLOYEE})$ 
```

Algorithm 1: Linear Search (Full Table Scan)

Working Steps

- 1 Read each block of relation R sequentially.
- 2 For every tuple t , evaluate the predicate.
- 3 Output t if it satisfies the condition.

I/O Cost

If b_R = number of blocks in relation:

$$\text{Cost} = b_R \text{ I/Os}$$

Examples

```
SELECT * FROM STUDENT WHERE dept='CSE';
```

All blocks are read sequentially and tested.

Advantages

- Works for any file type.
- Simple and sequential access.
- Good for small relations or no index.

Disadvantages

- Reads all blocks $\rightarrow$ high cost.
- Ignores selectivity of predicate.

Examples

Example: 2000 blocks $\rightarrow$ 2000 I/Os vs. Index Scan (4–5 I/Os)

Preconditions

- File sorted on the attribute used in predicate.
- Predicate is **equality** or simple **range**.

Procedure

- 1 Perform binary search on sorted attribute.
- 2 Locate first matching record.
- 3 Scan sequentially until condition fails.

Examples

```
SELECT * FROM EMPLOYEE WHERE emp_id = 105;
```

```
 $\Rightarrow \sigma_{\text{emp\_id}=105}(\text{Employee})$ 
```

Cost Estimate

$$\text{I/O Cost} = \log_2(b_R) + n_{\text{matching blocks}}$$

Example

For 1024 blocks and one matching block:

$$\text{Cost} = 10 + 1 = 11 \text{ I/Os}$$

Limitations

- Works only if data is physically sorted.
- Still sequentially scans range results.

Algorithm 3: Primary Index (Clustered Index Scan)

Idea

Use a **B⁺-tree or similar index** to locate records directly without scanning the entire relation.

Procedure

- 1 Traverse index (root $\rightarrow$ leaf).
- 2 Locate key(s) satisfying condition.
- 3 Fetch the data block(s) pointed by leaf entries.

Examples

`SELECT * FROM Employee WHERE emp_id=1003;` $\Rightarrow$ B⁺-tree lookup + 1 data block fetch.

Cost Formula

$$\text{Cost} = h + \# \text{ of data blocks retrieved}$$

where h = height of index.

Performance

- Clustered index contiguous data blocks.
- Great for equality and range predicates.

Example

If $h = 3$ and one data block needed, Cost = 4 I/Os (vs. 2000 for full scan).

Algorithm 4: Secondary (Non-Clustered) Index Scan

Concept

Index exists but data file is **unsorted** on that attribute.

Steps

- ① Use index to find matching record pointers (ROWIDs).
- ② Fetch each corresponding data block individually.

Cost

$$\text{Cost} = h + n_{\text{matching records}}$$

Each record fetch random I/O

Examples

100 HR employees, $h = 3$ $3 + 100 = 103$ I/Os

Advantages

- Fast for high-selectivity queries.
- Index on non-key attributes allowed.

Drawbacks

- Costly for low-selectivity (many matches).
- Random I/Os reduce performance.

Examples

```
SELECT * FROM Employee WHERE Dept='HR';
```

Useful only if few employees are in HR.

Idea

Use a **hash function** to locate the bucket containing the desired record.

Steps

- 1 Compute $h(\text{search_key})$.
- 2 Go to corresponding bucket.
- 3 Retrieve matching record(s).

Examples

```
SELECT * FROM Account WHERE AccNo=12345;  
Hash index on AccNo 1 bucket lookup.
```

Cost

Advantages

- Constant-time lookups ($O(1)$ average).
- Excellent for equality predicates.

Disadvantages

- Not suitable for range queries.
- Bucket overflow or dynamic hashing overhead.

Examples

WHERE AccNo=... **right** WHERE AccNo>1000 **wrong**

Conjunction (AND)

$$\sigma_{C_1 \wedge C_2}(R) = \sigma_{C_1}(\sigma_{C_2}(R))$$

Evaluate most **selective** condition first.

Disjunction (OR)

$$\sigma_{C_1 \vee C_2}(R) = \sigma_{C_1}(R) \cup \sigma_{C_2}(R)$$

If both attributes have indexes $\rightarrow$ perform union of result sets.

Negation (NOT)

Typically requires full scan since complement sets are not easily indexed.

Performance Summary

Algorithm	Works On	Predicate	Typical Cost
Linear Scan	Any	Any	b_R I/Os
Binary Search	Sorted File	Equality/Range	$\log_2(b_R) + k$
Primary Index	Clustered Index	Equality/Range	$h + 1$
Secondary Index	Non-Clustered Index	Equality	$h + n_{match}$
Hash Index	Hashed File	Equality	≈ 2

Decision Process

- 1 Analyze predicate type (equality, range, composite).
- 2 Check index availability.
- 3 Estimate selectivity and I/O cost.
- 4 Choose minimal-cost algorithm.

Examples

Example Decision Path:

If equality condition: → Hash index? use hash. → Else B⁺ index? use index lookup. Else
if range: → Clustered index? range scan. Else: → Full table scan.

Performance Summary

Algorithm	Cost Type	Remarks
Linear Scan	$O(b_R)$	Always works, slowest
Binary Search	$O(\log b_R)$	Requires sorted file
Index Scan	$O(\log_f N)$	Very efficient if index exists
Hash Search	$O(1)$ (avg)	Only for equality conditions
Composite Predicates	Varies	Combination of above

Examples

SQL Queries and Algorithms

SQL Query	Rel. Algebra	Likely Algorithm
SELECT * FROM Emp WHERE dept='HR';	$\sigma_{\text{dept}='HR'}(Emp)$	Linear / Index Scan
SELECT * FROM Emp WHERE id=103;	$\sigma_{\text{id}=103}(Emp)$	Binary / Index Lookup
SELECT * FROM Emp WHERE age>50;	$\sigma_{\text{age}>50}(Emp)$	Range Index Scan

Key Takeaways

- Linear scan is universal but inefficient.
- Binary and Index scans are faster with sorted data or indexes.
- DBMS optimizer chooses the most **cost-effective** algorithm.

Checkpoint Questions

- Why is binary search applicable only to sorted relations?
- Which algorithm is best suited for range queries?
- Why is index scan preferred for highly selective conditions?

Key Takeaways

- Selection retrieves tuples satisfying a condition.
- DBMS may use Linear, Binary, Index, or Hash methods.
- Optimizer picks path with lowest estimated I/O cost.

Practice Question

A relation EMP (2000 blocks) has a clustered index (height=3). Estimate I/Os for WHERE salary=50000.

Solution: $3 + 1 = 4$ I/Os (vs. 2000 for full scan).

- 1 Algorithms for External Sorting
- 2 Algorithms for SELECT Operations
- 3 Algorithms for JOIN Operations

Definition

The **Join operation** combines tuples from two relations based on a common attribute.

$$R_{R.A=S.B}S$$

Examples

```
SELECT *  
FROM Student JOIN Department  
ON Student.dept_id = Department.dept_id;  
⇒  $\pi_{\text{name, dept\_name}}(\text{Student}_{\text{dept\_id}} \text{Department})$ 
```

Goal

To find all matching pairs of tuples efficiently using minimal **I/O cost**.

DBMS Challenges

- Relations stored as separate disk files.
- Tuples must be compared by join attributes.
- Disk I/O dominates runtime.

Examples

Example:

Join Student(5000 tuples) with Dept(30 tuples).

Naïve nested loop 150,000 comparisons.

Optimized join few thousand I/Os.

Performance Objective

Minimize the number of **disk block reads/writes** using smart join strategies.

Key Factors

- **Relation size:** number of tuples and blocks in R and S.
- **File organization:** heap, sorted, indexed, hashed.
- **Join attribute:** equality vs range condition.
- **Buffer availability:** determines block loading.
- **Indexes:** clustered or secondary indexes.

Examples

Optimizer uses these statistics to choose among: Nested Loop, Block Nested Loop, Indexed NLJ, Sort-Merge, or Hash Join.

Algorithm 1: Simple Nested Loop Join (SNLJ)

Procedure

- 1 For each tuple $r \in R$
- 2 For each tuple $s \in S$
- 3 If $r.A = s.B$, output (r, s)

Cost

$$\text{I/O Cost} = b_R + (n_R \times b_S)$$

Examples

If $R = 100$ blocks, $S = 500$ blocks $\Rightarrow 100 + (100 \times 500) = 50,100$ I/Os

Advantages

- Works with any join condition.
- Simple and always valid.

Disadvantages

- Requires scanning S for every tuple of R .
- Cost grows as $O(|R| \times |S|)$.

Examples

Use only for **very small relations** or testing.

Algorithm 2: Block Nested Loop Join (BNLJ)

Concept

Process one or more blocks of R in memory at a time to reduce repeated scans of S .

Steps

- 1 Load a block (or set of $(B - 2)$ blocks) of R into memory.
- 2 For each block of S , compare with in-memory tuples.
- 3 Write matching pairs to output.

Cost Formula

$$\text{Cost} = b_R + \left\lceil \frac{b_R}{B - 2} \right\rceil \times b_S$$

Examples

$b_R = 100$, $b_S = 500$, $B = 5$ buffers:

$$\text{Cost} = 100 + \left\lceil \frac{100}{3} \right\rceil \times 500 = 100 + 17 \times 500 = 8,600 \text{ I/Os}$$

Advantages

- Utilizes buffer blocks effectively.
- Much faster than SNLJ.

Disadvantages

- Still costly if R large.
- Cannot use indexes.

Algorithm 3: Indexed Nested Loop Join (INLJ)

Idea

If inner relation S has an **index** on join attribute, use it for fast look-up.

Procedure

- 1 For each tuple $r \in R$,
- 2 Probe index on S using $r.A$,
- 3 Retrieve all matching tuples and output (r, s) .

Cost

$$\text{Cost} = b_R + (n_R \times C_{\text{index lookup}})$$

Examples

$R = 10,000$ tuples, $C_{\text{index}} = 3$ I/Os

$\Rightarrow b_R + (10,000 \times 3) = 30,000 + \text{few I/Os}$

Advantages

- Exploits existing index.
- Very efficient for high-selectivity joins.

Drawbacks

- Random I/O for each outer tuple.
- Poor if many matching tuples or no index.

Concept

Sort both relations on join key, then merge sequentially.

Procedure

- ① Sort R and S on join keys.
- ② Initialize pointers at first tuples.
- ③ Compare join keys:
 - If equal $\Rightarrow$ output.
 - Else advance pointer with smaller key.

Cost

$$\begin{aligned}\text{Cost} &= \text{Sort}(R) + \text{Sort}(S) + \text{Merge Phase} \\ &\approx 2(b_R \log b_R + b_S \log b_S) + (b_R + b_S)\end{aligned}$$

Examples

Relations already sorted on dept_id:

R.dept_id	R.name	S.dept_id	S.dept_name
1	A	1	CSE
2	B	2	ECE
3	C	3	ME

Advantages

- Excellent for sorted inputs.
- Supports range and outer joins.

Limitations

- High sorting cost if unsorted.
- Requires sequential access.

Concept

Partition both relations using same hash function and join matching partitions.

Phases

- 1 **Build Phase:** Hash smaller relation R into partitions.
- 2 **Probe Phase:** For each tuple in S , hash on same key and probe partition.

Cost

Each partition read, written, then re-read:

$$\text{Cost} = 3(b_R + b_S)$$

Examples

Student Department on dept_id

→ Use $h(\text{dept_id})$ to partition both relations.

→ Join each pair of partitions in memory.

Advantages

- Very efficient for large unsorted data.
- Linear I/O complexity.
- Easy to parallelize.

Limitations

- Not suitable for range joins.
- Needs sufficient memory for partitions.

Selection Strategy

- If inner has index → use **Indexed Nested Loop**.
- If inputs are sorted → use **Sort-Merge Join**.
- If large unsorted → use **Hash Join**.
- Otherwise → use **Block Nested Loop**.

Examples

Given $R = 200$, $S = 800$, $B = 20$:

BNLJ	7,400 I/Os
Sort-Merge	$\approx 3,200$ I/Os
Hash Join	3,000 I/Os

Hash Join is optimal.

Performance Summary

Algorithm	Sorted?	Index?	Cost (I/Os)	Best For
Simple NLJ	No	No	$b_R \times b_S$	Small relations
Block NLJ	No	No	$b_R + (b_R / (B - 2)) \times b_S$	Medium data
Indexed NLJ	Inner indexed	Yes	$b_R + n_R \times C_{idx}$	Selective joins
Sort-Merge	Yes	No	Sort + Merge	Sorted or range joins
Hash Join	No	No	$3(b_R + b_S)$	Large unsorted data

Key Takeaways

- All join algorithms aim to minimize **disk I/O**.
- **Hash Join** usually best for large unsorted data.
- **Sort-Merge Join** ideal for sorted or range joins.
- **Indexed Join** wins when index available.

Practice

- 1 Why is Hash Join unsuitable for range conditions?
- 2 How does Block NLJ exploit extra memory?
- 3 Estimate cost for $b_R = 1000$, $b_S = 2000$, $B = 20$.

Thank You!



Algorithms for PROJECT, SET, Aggregate Operations

DBMS Lecture Series

Unit-4

Dr. Pavinder Yadav

Assistant Professor
School of Computer Science
University of Petroleum and Energy Studies
Dehradun



- 1 Algorithms for PROJECT Operations
- 2 Algorithms for SET Operations
- 3 Implementing Aggregate Operations and Outer Joins
- 4 Summary of Query Processing Algorithms

- 1 Algorithms for PROJECT Operations
- 2 Algorithms for SET Operations
- 3 Implementing Aggregate Operations and Outer Joins
- 4 Summary of Query Processing Algorithms

Objective

Understand how a DBMS implements:

- The **PROJECT** operation (attribute selection)
- The **SET operations**: UNION, INTERSECT, and DIFFERENCE

Examples

`SELECT DISTINCT dept_id FROM Student;` $\Rightarrow$ Projection without duplicates.

`SELECT * FROM A UNION SELECT * FROM B;` $\Rightarrow$ Set union.

Importance

Projection and Set operations appear frequently in query execution plans and require efficient handling of duplicates.

Definition

The **Projection operator** extracts selected columns from a relation:

$$\pi_{A_1, A_2, \dots, A_k}(R)$$

Examples

```
SELECT name, dept_id FROM Student;
```

$\Rightarrow \pi_{\text{name, dept_id}}(\text{Student})$

Two Variants

- **With duplicates:** Keeps all tuples (default SQL behavior)
- **Without duplicates:** Eliminates repetitions using sorting or hashing

Procedure

- 1 Scan each tuple of R .
- 2 Output only required attributes.
- 3 Do not check for duplicates.

Cost

$$\text{Cost} = b_R$$

(One sequential scan of relation R)

Examples

```
SELECT name, dept_id FROM Student;
```

Outputs selected columns quickly but keeps duplicates.

Procedure

- 1 Project only required attributes.
- 2 Sort the resulting tuples on projected attributes.
- 3 Scan sorted output and remove duplicates.

Cost

If external merge sort used:

$$\text{Cost} = 2b_R \log_{B-1}(b_R) + b_R$$

where B = number of buffers.

Examples

```
SELECT DISTINCT dept_id FROM Student;
```

Procedure

- 1 Apply a hash function on projected attributes.
- 2 Partition tuples into hash buckets.
- 3 Within each bucket, eliminate duplicates using in-memory hash tables.

Cost

$$\text{Cost} \approx 3b_R$$

(1 read + 1 write for partitioning, +1 read for output)

Remarks

- Faster than sorting for large unsorted data.
- Produces no ordering in output.

Performance Summary

Method	Duplicates Removed?	Cost	Remarks
Simple Scan	No	b_R	Fastest, no ordering
Sort-Based	Yes	$2b_R \log(b_R) + b_R$	Produces ordered output
Hash-Based	Yes	$3b_R$	Better for large unsorted data

- 1 Algorithms for PROJECT Operations
- 2 Algorithms for SET Operations
- 3 Implementing Aggregate Operations and Outer Joins
- 4 Summary of Query Processing Algorithms

SQL and Relational Algebra

SQL Operator	Relational Algebra	Meaning
UNION	$\cup$	Tuples from both relations (unique)
INTERSECT	$\cap$	Tuples present in both
EXCEPT / MINUS	$-$	Tuples in first, not in second

Constraints

Relations must have the **same schema and data types**. Duplicates removed unless UNION ALL used.

General Steps

- ① Sort both relations R and S on all attributes.
- ② Scan sorted results together:
 - **UNION:** Merge unique tuples.
 - **INTERSECT:** Output only matching tuples.
 - **DIFFERENCE:** Output tuples in R not in S .

Cost

$$\text{Cost} = 2(b_R \log b_R + b_S \log b_S) + (b_R + b_S)$$

Examples

R	S
1	2
2	3
3	4

- UNION = 1,2,3,4
- INTERSECT = 2,3
- DIFFERENCE (R-S) = 1

Advantages

- Handles duplicates naturally.
- Suitable when input already sorted.

Procedure

- ① Partition both R and S using same hash function $h()$.
- ② For each pair of partitions (R_i, S_i) :
 - **UNION:** Add all unique tuples from both.
 - **INTERSECT:** Add tuples found in both.
 - **DIFFERENCE:** Add tuples in R_i not in S_i .

Cost

$$\text{Cost} \approx 3(b_R + b_S)$$

Concept

UNION ALL simply concatenates two relations without duplicate removal.

Cost

$$\text{Cost} = b_R + b_S$$

No sorting or hashing required.

Examples

```
SELECT * FROM Sales2024
UNION ALL
SELECT * FROM Sales2025;
```

Performance Summary

Operation	Sorting?	Hashing?	Cost	Remarks
UNION	Yes	Yes	$2b_R \log b_R + 2b_S \log b_S$	Removes duplicates
UNION ALL	No	No	$b_R + b_S$	Keeps duplicates
INTERSECT	Yes	Yes	$3(b_R + b_S)$	Uses equality checks
DIFFERENCE	Yes	Yes	$3(b_R + b_S)$	Subtract matching tuples

Strategy

- If inputs already sorted $\rightarrow$ **Sort-based algorithms.**
- Else if unsorted and large $\rightarrow$ **Hash-based algorithms.**
- For UNION ALL $\rightarrow$ simple concatenation.

Examples

If R and S are 10,000 blocks unsorted:

Sort-based cost high ($\sim \log$ terms),

Hash-based cost $3(10,000 + 10,000) = 60,000$ I/Os $\rightarrow$ faster.

Key Takeaways

- **Projection:** Column selection, duplicates optional.
- **Set operations:** Combine or compare relations.
- Sort-based = ordered output, higher cost.
- Hash-based = faster, unordered.
- Optimizer picks lowest estimated cost.

Practice Questions

- 1 Estimate I/O cost for projecting 2000 blocks using sort-based method.
- 2 Which is faster for UNION on large unsorted data?
- 3 Explain how UNION ALL differs in cost from UNION.

- 1 Algorithms for PROJECT Operations
- 2 Algorithms for SET Operations
- 3 Implementing Aggregate Operations and Outer Joins
- 4 Summary of Query Processing Algorithms

Learning Objectives

After this topic, you will be able to:

- Understand how DBMS computes **aggregate functions**.
- Explain **sorting-based** and **hash-based** aggregation.
- Learn how **outer joins** are implemented internally.

Examples

Aggregates: SUM, COUNT, AVG, MIN, MAX

Outer Joins: LEFT, RIGHT, FULL

Definition

Aggregate functions summarize data over groups of tuples.

Function	Description
SUM	Adds up values
COUNT	Counts number of tuples
AVG	Computes average
MIN	Minimum value
MAX	Maximum value

Examples

```
SELECT AVG(salary) FROM Employee; ⇒ One output value
```

```
SELECT dept_id, SUM(salary) FROM Employee GROUP BY dept_id;
```

```
⇒ One tuple per group
```

Three Stages

- ➊ **Grouping Phase:** Tuples divided by GROUP BY attributes.
- ➋ **Computation Phase:** Apply functions (SUM, AVG, etc.).
- ➌ **Output Phase:** Produce one result tuple per group.

Examples

```
SELECT dept_id, AVG(salary)
FROM Employee GROUP BY dept_id;
```

Goal

Perform grouping and aggregation with minimal I/O cost.

Idea

Sort tuples on grouping attribute and compute aggregates in one scan.

Procedure

- 1 Sort relation R on grouping attribute(s).
- 2 Initialize counters for each aggregate.
- 3 When group changes, output computed aggregates.

Examples

```
SELECT dept_id, SUM(salary)
FROM Employee GROUP BY dept_id;
```

Examples

After sorting on dept_id:

dept_id	salary
10	50000
10	70000
20	40000

Output: 10 → 120000, 20 → 40000

Cost

$$\text{Cost} = \text{Sort}(R) + \text{Scan}(R)$$

If external sort used:

$$\approx 2b_R \log_{B-1}(b_R) + b_R$$

Remarks

Efficient when data already sorted or ORDER BY clause used.

Idea

Use a hash function to divide tuples into groups; compute aggregates per hash bucket.

Procedure

- 1 Apply hash function $h(\text{group_attr})$.
- 2 Place tuples into corresponding hash buckets.
- 3 Maintain partial aggregates (SUM, COUNT, etc.) in memory.
- 4 Output final aggregates for each bucket.

Cost

$$\text{Cost} \approx 3b_R$$

(1 read + 1 write for partitioning + 1 read for aggregation)

Examples

```
SELECT dept_id, COUNT(*) FROM Employee GROUP BY dept_id;
```

Tuples with same dept_id $\Rightarrow$ same hash bucket.

Count maintained per bucket.

Comparison

Method	Sorted Input?	Cost	Remarks
Sorting-Based	Yes	$2b_R \log(b_R) + b_R$	Produces ordered output
Hash-Based	No	$3b_R$	Faster for large unsorted data

Optimizations in Query Engine

- **Early aggregation:** Perform partial aggregation before joins.
- **Push-down aggregation:** Move aggregation closer to data source.
- Combine ORDER BY sorting with aggregation when possible.

Examples

```
SELECT dept_id, COUNT(*) FROM Emp GROUP BY dept_id;  
→ DBMS may pre-aggregate by dept_id before join or sort.
```


Definition

Outer Join preserves unmatched tuples by padding them with **NULL** values.

Type	Description
Left Outer Join	Keeps all tuples from left relation
Right Outer Join	Keeps all tuples from right relation
Full Outer Join	Keeps all tuples from both relations

Examples

```
SELECT E.name, D.dept_name  
FROM Employee E LEFT JOIN Department D  
ON E.dept_id = D.dept_id;
```

Sort-Merge Outer Join

- 1 Sort both R and S on join key.
- 2 Perform merge like inner join.
- 3 For unmatched tuples, output NULL for missing side.

Hash-Based Outer Join

- 1 Build hash table on smaller relation.
- 2 Probe larger relation for matches.
- 3 Output non-matching tuples with NULLs.

Examples

Employee

emp_id	name	dept_id
1	Alice	10
2	Bob	20

Department

dept_id	dept_name
10	CSE

Result (Left Outer Join):

name	dept_name
Alice	CSE
Bob	NULL

Cost Estimate

Sort-Merge Outer Join $\approx \text{Sort}(R) + \text{Sort}(S) + \text{Merge}$

Hash Outer Join $\approx 3(b_R + b_S)$

Plus additional cost for unmatched tuples.

Algorithm Choice

- **Sort-Merge:** Best when inputs already sorted.
- **Hash Join:** Best for large, unsorted data.

Aggregations

- **Sorting-based:** Uses external sort, produces ordered results.
- **Hash-based:** Uses hash buckets, faster for unsorted data.

Outer Joins

- Include unmatched tuples using NULL padding.
- Implemented via Sort-Merge or Hash Join methods.

Cost Overview

Operation	Approximate Cost
Sort Aggregation	$2b_R \log(b_R) + b_R$
Hash Aggregation	$3b_R$
Outer Join (Hash)	$3(b_R + b_S)$

Check Your Understanding

- 1 Explain difference between Sort and Hash Aggregation.
- 2 How does DBMS handle unmatched tuples in Outer Join?
- 3 When is Hash Aggregation preferred over Sorting?

Examples

Example Question:

Estimate cost of aggregating 2000 blocks using sort-based method with 10 buffers.

$\Rightarrow \approx 2(2000) \log_9(2000) + 2000 \text{ I/Os.}$

- 1 Algorithms for PROJECT Operations
- 2 Algorithms for SET Operations
- 3 Implementing Aggregate Operations and Outer Joins
- 4 Summary of Query Processing Algorithms**

1. Selection (σ) Operation

Algorithms:

- **Linear Scan:** Cost = b_R
- **Binary Search:** Cost = $\log_2(b_R)$ (if relation sorted)
- **Indexed Access:** Cost = $C_{\text{index lookup}}$

Choice

Use index if selection condition is **highly selective**; otherwise use sequential scan.

2. Join (⋈) Operation

Common Join Algorithms and I/O Cost:

Simple NLJ: $b_R + (n_R \times b_S)$

Block NLJ: $b_R + \left\lceil \frac{b_R}{B-2} \right\rceil \times b_S$

Indexed NLJ: $b_R + n_R \times C_{\text{index}}$

Sort-Merge Join: $\text{Sort}(R) + \text{Sort}(S) + \text{Merge}$

Hash Join: $3(b_R + b_S)$

Best Choices:

- Sorted relations $\Rightarrow$ Sort-Merge Join
- Unsorted large data $\Rightarrow$ Hash Join
- Indexed inner relation $\Rightarrow$ Indexed NLJ

3. Projection (π) Operation

Algorithms:

Simple Projection: b_R

Sort-based: $2b_R \log_{B-1}(b_R) + b_R$

Hash-based: $3b_R$

Examples

Remarks: Hash faster for large unsorted data; sort-based gives ordered output.

4. Set Operations ($\cup$, $\cap$, $-$)

Algorithms and Cost:

Sort-based: $2(b_R \log b_R + b_S \log b_S) + (b_R + b_S)$

Hash-based: $3(b_R + b_S)$

UNION ALL: $b_R + b_S$

Choice: Sorted data $\Rightarrow$ Sort-based; Unsorted data $\Rightarrow$ Hash-based.

5. Aggregation & Outer Join

Sort Aggregation: $2b_R \log(b_R) + b_R$

Hash Aggregation: $3b_R$

Outer Join (Hash/Sort): $\approx 3(b_R + b_S)$

Rule: Hash methods $\Rightarrow$ Faster; Sort-based $\Rightarrow$ Ordered output.

Thank You!

